

Week 3 - part 2

CS 4200 / 6200

Linear Algebra for QC

Nella Ludlow, PhD
Wright State University
Week 3

Begin of Matrix Multiplication Review

Matrix Multiplication

- ▶ Another basic matrix operation is **matrix multiplication**. You will see later, that this definition of the product of two matrices has many practical applications.

Definition of Matrix Multiplication

If $A = [a_{ij}]$ is an $m \times n$ matrix and $B = [b_{ij}]$ is an $n \times p$ matrix, then the product AB is an $m \times p$ matrix given by

$$AB = [c_{ij}]$$

where $c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + a_{i3}b_{3j} + \cdots + a_{in}b_{nj}$.

Matrix Multiplication

- ▶ The definition of matrix multiplication indicates a *row-by-column multiplication*, where the entry in the i th row and j th column of the product AB is obtained by multiplying the entries in the i th row A of by the corresponding entries in the j th column of B and then adding the results.

Matrix Multiplication

- The general pattern for matrix multiplication is as follows.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \dots & a_{in} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1j} & \dots & b_{1p} \\ b_{21} & b_{22} & \dots & b_{2j} & \dots & b_{2p} \\ b_{31} & b_{32} & \dots & b_{3j} & \dots & b_{3p} \\ \vdots & \vdots & & \vdots & & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nj} & \dots & b_{np} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1j} & \dots & c_{1p} \\ c_{21} & c_{22} & \dots & c_{2j} & \dots & c_{2p} \\ \vdots & \vdots & & \vdots & & \vdots \\ c_{i1} & c_{i2} & \dots & c_{ij} & \dots & c_{ip} \\ \vdots & \vdots & & \vdots & & \vdots \\ c_{m1} & c_{m2} & \dots & c_{mj} & \dots & c_{mp} \end{bmatrix}$$

$a_{i1}b_{1j} + a_{i2}b_{2j} + a_{i3}b_{3j} + \dots + a_{in}b_{nj} = c_{ij}$

Example 7 - Finding the Product of Two Matrices

► Find the product AB using $A = \begin{bmatrix} -1 & 3 \\ 4 & -2 \\ 5 & 0 \end{bmatrix}$ and $B = \begin{bmatrix} -3 & 2 \\ -4 & 1 \end{bmatrix}$.

► Solution:

►

First, note that the product is defined because the number of columns of A is equal to the number of rows of B . Moreover, the product AB has dimension 3×2 .

To find the entries of the product, multiply each row of A by each column of B .

Example 7 - *Solution*

cont'd

$$\blacktriangleright \quad AB = \begin{bmatrix} -1 & 3 \\ 4 & -2 \\ 5 & 0 \end{bmatrix} \begin{bmatrix} -3 & 2 \\ -4 & 1 \end{bmatrix}$$

$$\blacktriangleright \quad = \begin{bmatrix} (-1)(-3) + (3)(-4) & (-1)(2) + (3)(1) \\ (4)(-3) + (-2)(-4) & (4)(2) + (-2)(1) \\ (5)(-3) + (0)(-4) & (5)(2) + (0)(1) \end{bmatrix}$$

$$\blacktriangleright \quad = \begin{bmatrix} -9 & 1 \\ -4 & 6 \\ -15 & 10 \end{bmatrix}$$

Matrix Multiplication

- ▶ The general pattern for matrix multiplication is as follows.

Properties of Matrix Multiplication

Let A , B , and C be matrices and let c be a scalar.

1. $A(BC) = (AB)C$

Associative Property of Matrix Multiplication

2. $A(B + C) = AB + AC$

Left Distributive Property

3. $(A + B)C = AC + BC$

Right Distributive Property

4. $c(AB) = (cA)B = A(cB)$

Associative Property of Scalar Multiplication

Matrix Multiplication

Definition of Identity Matrix

The $n \times n$ matrix that consists of 1's on its main diagonal and 0's elsewhere is called the **identity matrix of dimension $n \times n$** and is denoted by

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}. \quad \text{Identity matrix}$$

Note that an identity matrix must be *square*. When the dimension is understood to be $n \times n$, you can denote I_n simply by I .

Matrix Multiplication

- ▶ If A is an $n \times n$ matrix, then the identity matrix has the property that $AI_n = A$ and $I_n A = A$.
- ▶ For example,

$$\begin{bmatrix} 3 & -2 & 5 \\ 1 & 0 & 4 \\ -1 & 2 & -3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & -2 & 5 \\ 1 & 0 & 4 \\ -1 & 2 & -3 \end{bmatrix} \quad AI = A$$

End of Matrix Multiplication Review

Algorithm Complexity - Matrix Multiplication

Where are we on this scale?

1-1 Comparison of running times

For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem takes $f(n)$ microseconds.

	second	minute	hour	day	month	year	century
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							

Fastest!

Fast!

Linear!

Merge Sort->

Most Sorts->

Slow!

Super slow!

Intractable!



The traditional way:

$$C_{ij} = \sum_{k=1}^n A_{ik} \times B_{kj}$$

use three for-loop

$$\therefore T(n) = O(n^3)$$

The Divide-and-Conquer way:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{11} = \underline{A_{11} \cdot B_{11}} + \underline{A_{12} \cdot B_{21}}$$

$$C_{12} = \underline{A_{11} \cdot B_{12}} + \underline{A_{12} \cdot B_{22}}$$

$$C_{21} = \underline{A_{21} \cdot B_{11}} + \underline{A_{22} \cdot B_{21}}$$

$$C_{22} = \underline{A_{21} \cdot B_{12}} + \underline{A_{22} \cdot B_{22}}$$

transform the problem of multiplying A and B , each of size $[n \times n]$
into 8 subproblems, each of size $\left[\frac{n}{2} \times \frac{n}{2}\right]$

$$\begin{aligned}\therefore T(n) &= 8 \cdot T\left(\frac{n}{2}\right) + an^2 \\ &= O(n^3)\end{aligned}$$

which an^2 is for addition

so, it is no improvement compared with the traditional way

Example:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

use Divide-and-Conquer way to solve it as following:

$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 3 & 3 & 0 \\ 3 & 3 & 3 & 0 \\ 3 & 3 & 3 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$C_{11} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix}$$

$$C_{12} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 3 & 0 \end{bmatrix}$$

$$C_{21} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 3 \\ 0 & 0 \end{bmatrix}$$

$$C_{22} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$$

Strassen's matrix multiplication:

- Discover a way to compute the C_{ij} 's using 7 multiplications and 18 additions or subtractions

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

•Algorithm :

procedure Strassen (n, A, B, C) // n is size, A, B the input
matrices, C output matrix

begin

if $n = 2$,

$$\left\{ \begin{array}{l} C_{11} = a_{11} \cdot b_{11} + a_{12} \cdot b_{21}; \\ C_{12} = a_{11} \cdot b_{12} + a_{12} \cdot b_{22}; \\ C_{21} = a_{21} \cdot b_{11} + a_{22} \cdot b_{21}; \\ C_{22} = a_{21} \cdot b_{12} + a_{22} \cdot b_{22}; \end{array} \right.$$

else

(cont.)

else

Partition A into 4 submatrices: $A_{11}, A_{12}, A_{21}, A_{22}$;

Partition B into 4 submatrices: $B_{11}, B_{12}, B_{21}, B_{22}$;

call Strassen ($\frac{n}{2}, A_{11} + A_{22}, B_{11} + B_{22}, P$);

call Strassen ($\frac{n}{2}, A_{21} + A_{22}, B_{11}, Q$);

call Strassen ($\frac{n}{2}, A_{11}, B_{12} - B_{22}, R$);

call Strassen ($\frac{n}{2}, A_{22}, B_{21} - B_{11}, S$);

call Strassen ($\frac{n}{2}, A_{11} + A_{12}, B_{22}, T$);

call Strassen ($\frac{n}{2}, A_{21} - A_{11}, B_{11} + B_{12}, U$);

call Strassen ($\frac{n}{2}, A_{12} - A_{22}, B_{21} + B_{22}, V$);

$$C_{11} = P + S - T + V;$$

$$C_{12} = R + T;$$

$$C_{21} = Q + S;$$

$$C_{22} = P + R - Q + U;$$

end;

Analysis:

$$T(n) = \begin{cases} 7 \cdot T(\frac{n}{2}) + an^2 & n > 2 \\ b & n \leq 2 \end{cases}$$

$$T(n) = 7 \cdot T(\frac{n}{2}) + an^2$$

$$= 7^2 \cdot T(\frac{n}{2^2}) + (\frac{7}{4})an^2 + an^2$$

$$= 7^3 \cdot T(\frac{n}{2^3}) + (\frac{7}{4})^2 \cdot an^2 + (\frac{7}{4}) \cdot an^2 + an^2$$

...

Assume $n = 2^k$ for some integer k

$$\begin{aligned} &= 7^{k-1} \cdot T\left(\frac{n}{2^{k-1}}\right) + an^2 \cdot \left[\left(\frac{7}{4}\right)^{k-2} + \dots + 1 \right] \\ &= 7^{k-1} \cdot b + an^2 \left[\frac{\left(\frac{7}{4}\right)^{k-1} - 1}{\frac{7}{4} - 1} \right] \\ &\leq b \cdot 7^k + c \cdot n^2 \cdot \left(\frac{7}{4}\right)^k \\ &= b \cdot 7^{Lgn} + cn^2 \cdot \left(\frac{7}{4}\right)^{Lgn} = b \cdot 7^{Lgn} + cn^2 (n)^{Lg \frac{7}{4}} \\ &= b \cdot n^{Lg 7} + cn^{Lg 7} = (b + c) \cdot n^{Lg 7} \\ &= O(n^{Lg 7}) = O(n^{2.81}) \end{aligned}$$

Strassen Algorithm

► <https://www.youtube.com/watch?v=OSelh06Qnlc>

Strassen algorithm for matrix multiplication (divide and conquer) - Inside code

$$\begin{aligned}ae + bg &= (a + d)(e + h) + a(g - e) - (a + b)h + (b - d)(g + h) \\af + bh &= a(f - h) + (a + b)h \\ce + dg &= (c + d)e + d(g - e) \\cf + dh &= a(f - h) + (a + d)(e + h) - (c + d)e - (a - c)(e + f)\end{aligned}$$
$$\begin{aligned}c_{11} &= ae + bg = p_1 + p_2 - p_3 + p_4 \\c_{12} &= af + bh = p_5 + p_3 \\c_{21} &= ce + dg = p_6 + p_2 \\c_{22} &= cf + dh = p_5 + p_1 - p_6 - p_7\end{aligned}$$
$$\begin{aligned}p_1 &= (a + d)(e + h) & p_5 &= a(f - h) \\p_2 &= d(g - e) & p_6 &= (c + d)e \\p_3 &= (a + b)h & p_7 &= (a - c)(e + f) \\p_4 &= (b - d)(g + h)\end{aligned}$$

7:45 / 9:40

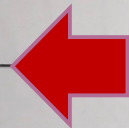
Algorithm Complexity - Matrix Multiplication

Where are we on this scale?

1-1 Comparison of running times

For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem takes $f(n)$ microseconds.

	second	minute	hour	day	month	year	century
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							



complexity of Strassen Algorithm is $O(n^{\lg 7})$ or ~ 2.807

Fastest!

Fast!

Linear!

Merge Sort->

Most Sorts->

Slow!

Super slow!

Intractable!

Wikipedia - Matrix Multiplication Algos

https://en.wikipedia.org/wiki/Matrix_multiplication_algorithm

Sub-cubic algorithms [\[edit\]](#)

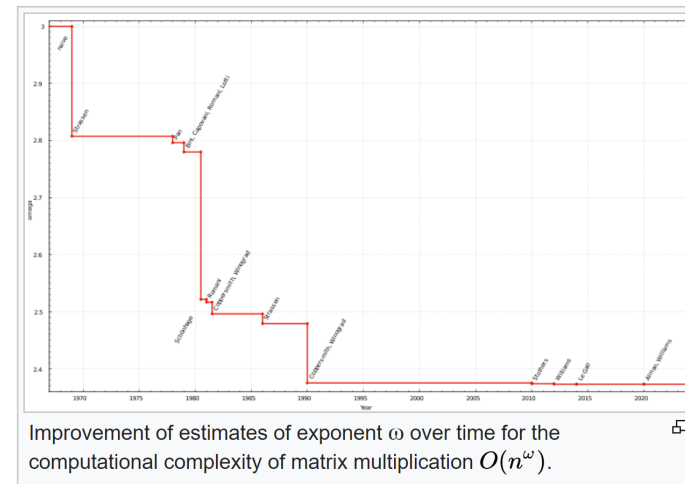
Further information: [Computational complexity of matrix multiplication](#)

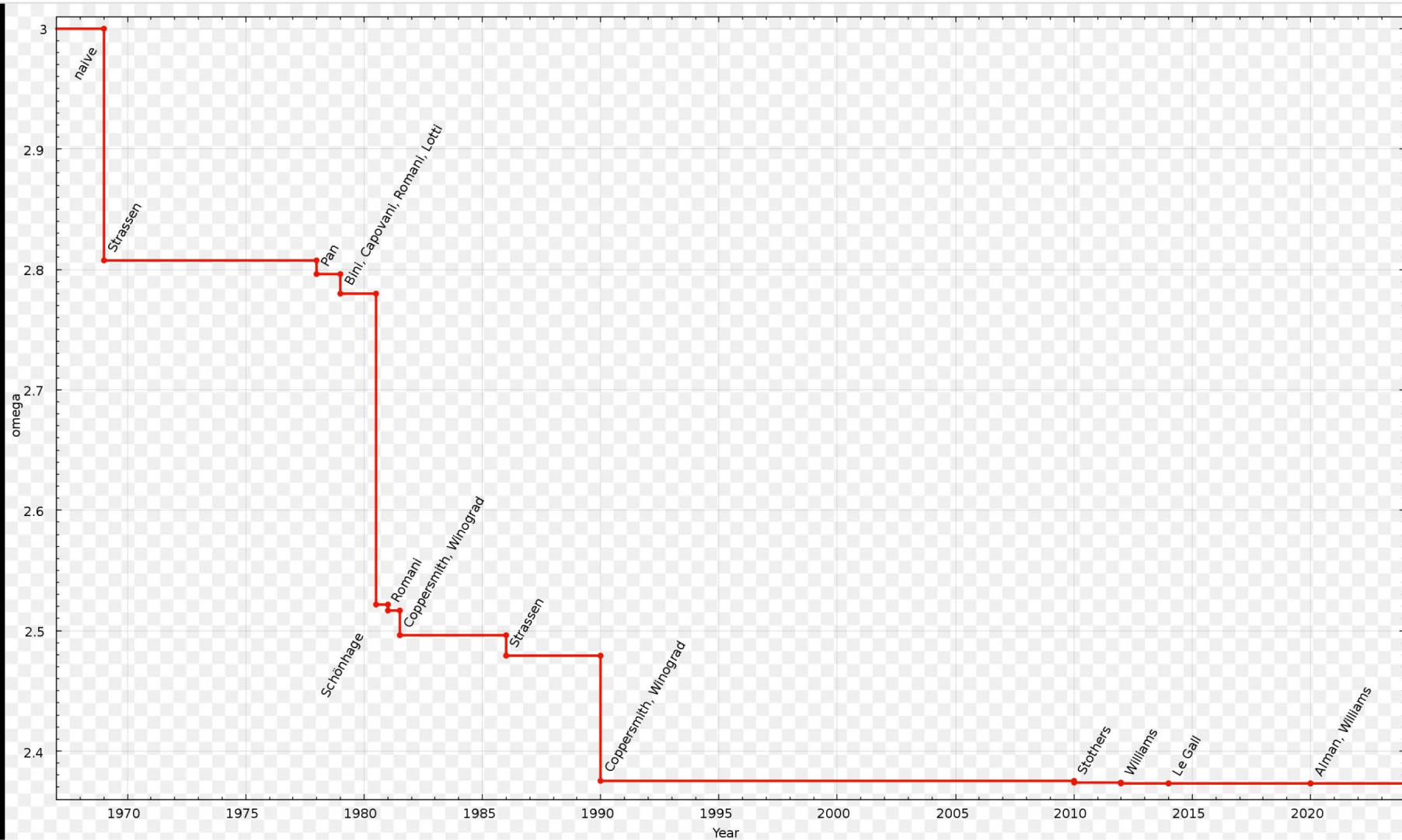
Algorithms exist that provide better running times than the straightforward ones. The first to be discovered was [Strassen's algorithm](#), devised by [Volker Strassen](#) in 1969 and often referred to as "fast matrix multiplication". It is based on a way of multiplying two 2×2 -matrices which requires only 7 multiplications (instead of the usual 8), at the expense of several additional addition and subtraction operations. Applying this recursively gives an algorithm with a multiplicative cost of $O(n^{\log_2 7}) \approx O(n^{2.807})$. Strassen's algorithm is more complex, and the [numerical stability](#) is reduced compared to the naïve algorithm,^[8] but it is faster in cases where $n > 100$ or so^[1] and appears in several libraries, such as [BLAS](#).^[9] It is very useful for large matrices over exact domains such as [finite fields](#), where numerical stability is not an issue.

It is an open question in [theoretical computer science](#) how well Strassen's algorithm can be improved in terms of [asymptotic complexity](#). The *matrix multiplication exponent*, usually denoted ω , is the smallest real number for which any $n \times n$ matrix over a field can be multiplied together using $n^{\omega+o(1)}$ field operations. The current best bound on ω is $\omega < 2.3728596$, by Josh Alman and [Virginia Vassilevska Williams](#).^[2] This algorithm, like all other recent algorithms in this line of research, is a generalization of the Coppersmith–Winograd algorithm, which was given by [Don Coppersmith](#) and [Shmuel Winograd](#) in 1990.^[10] The conceptual idea of these algorithms are similar to Strassen's algorithm: a way is devised for multiplying two $k \times k$ -matrices with fewer than k^3 multiplications, and this technique is applied recursively. However, the constant coefficient hidden by the [Big O notation](#) is so large that these algorithms are only worthwhile for matrices that are too large to handle on present-day computers.^{[11][12]}

[Freivalds' algorithm](#) is a simple [Monte Carlo algorithm](#) that, given matrices A , B and C , verifies in $\Theta(n^2)$ time if $AB = C$.

Parallel and distributed algorithms [\[edit\]](#)





Improvement of estimates of exponent ω over time for the computational complexity of matrix multiplication .

Professor Nella Ludlow - Wright State University

Jochen Burghardt - Own work

[More details](#)

CC BY-SA 4.0

Preview of Coming Attractions

- ▶ Tensors - how is that different than Matrix Multiplication?